Captured `self`	**Stale capture**
Body-local `let`	**Eager resolution**
Lazy resolution	**`??` operator**
Re-render	**Sibling bug**

A captured self whose values no longer match the live self -- the closure is reading from an outdated snapshot. Example: A save-on-dismiss closure captured self before the user's last edit landed, so it saves the wrong value.	When a closure references properties of the enclosing struct, Swift attaches a copy of self to the closure so it can read those properties later. Example: A SwiftUI onChange or task closure that reads self.itemCount is capturing self, not just that one property.
Doing the computation NOW, before the closure is built, so the closure captures the result instead of self. Example: Computing let id = item.id above a closure so the closure captures id, not a stale self.item.	A constant declared inside var body (or a builder scope) that captures a computed value at body-time. Example: let displayName = item.name inside body freezes that value for this render pass, before any closure runs.
Nil-coalescing: lhs ?? rhs returns lhs if it's non-nil, otherwise rhs. The closure-literal RHS is constructed at the same time as the surrounding expression. Example: let name = item.nickname ?? item.defaultName avoids an explicit if-let just to supply a fallback.	Doing the computation LATER, inside the closure, when it actually runs -- this is what causes staleness. Example: A closure that reads self.item.id when it fires, instead of when it was created, can read a value that's already changed.
A bug in a different file or function that shares the exact same root cause as a known bug. Example: After fixing one stale-capture bug, a bug-echo-style sweep for the same closure shape found two more in unrelated views.	When SwiftUI calls body again to recompute the view tree -- happens often, anytime watched state changes. Example: Every re-render re-runs body-local lets, which is exactly why eager resolution there stays fresh.